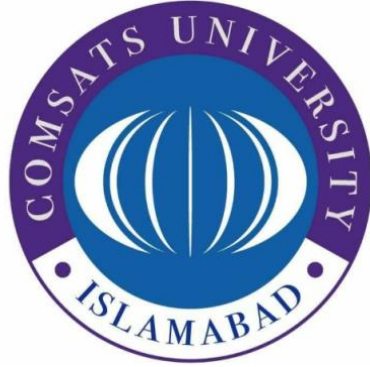


COMSATS UNIVERSITY ISLAMABAD LAHORE CAMPUS



Ahmad Shahzad

SP26-BAI-007

Object Oriented Programme

Dr Shahid Bhatti

Sep 19 2026

Task 1 - Build a Class and Prove Independent Object State

Student.java

```
AIExperiment - Notepad
File Edit Format View Help
class AIExperiment {
    String experimentName;
    int completedEpochs;
    int targetEpochs;

    void runEpochs(int epochs) {
        completedEpochs = completedEpochs + epochs;
    }

    void runEpochs(int epochs, int bonusEpochs) {
        int totalEpochs = epochs + bonusEpochs;
        completedEpochs = completedEpochs + totalEpochs;
    }

    int remainingEpochs() {
        return targetEpochs - completedEpochs;
    }

    String status() {
        return experimentName + ": " + completedEpochs + "/" + targetEpochs + " epochs done";
    }
}

Ln 1, Col 1    100%    Unix (LF)    UTF-8
```

Task1Demo.java

Output

```
AIExperimentDemo - Notepad
File Edit Format View Help
class AIExperimentDemo {

    void boostExperiment(AIExperiment exp, int bonus) {
        exp.completedEpochs = exp.completedEpochs + bonus;
    }

    public static void main(String[] args) {
        AIExperimentDemo helper = new AIExperimentDemo();

        AIExperiment exp1 = new AIExperiment();
        exp1.experimentName = "ImageClassifier";
        exp1.completedEpochs = 0;
        exp1.targetEpochs = 20;

        AIExperiment exp2 = new AIExperiment();
        exp2.experimentName = "TextSummarizer";
        exp2.completedEpochs = 0;
        exp2.targetEpochs = 15;

        System.out.println("Initial state:");
        System.out.println(exp1.status());
        System.out.println(exp2.status());
        System.out.println();

        exp1.runEpochs(5);
        exp2.runEpochs(3, 2);

        System.out.println("After runEpochs calls:");
        System.out.println(exp1.status());
        System.out.println("Remaining for exp1: " + exp1.remainingEpochs());
        System.out.println(exp2.status());
        System.out.println("Remaining for exp2: " + exp2.remainingEpochs());
        System.out.println();

        System.out.println("Only exp1 changes below:");
        exp1.runEpochs(2);
    }
}

Ln 1, Col 1    100%    Unix (LF)    UTF-8
```

Before change:
SP26-BAI-001 Abeer Amina 15
SP26-BAI-002 Ali Ishtiaq 18
SP26-BAI-003 Abdul Rehman Azam 12

After change (only s2 updated):
SP26-BAI-001 Abeer Amina 15
SP26-BAI-002 Ali Ishtiaq 21
SP26-BAI-003 Abdul Rehman Azam 12

1. What is the difference between the Student class and the objects s1, s2, and s3 created from it?

Answer: Student is just the blueprint/design, it doesn't hold any real data by itself. s1, s2 and s3 are actual objects made from that blueprint, and each one has its own real memory to store values.

2. Why does changing s2.completedCredits not change the completedCredits value stored in s1 or s3?

Answer: Each object gets its own separate copy of the instance variables when it is created with new, so s1, s2 and s3 each have their own completedCredits stored in different memory locations.

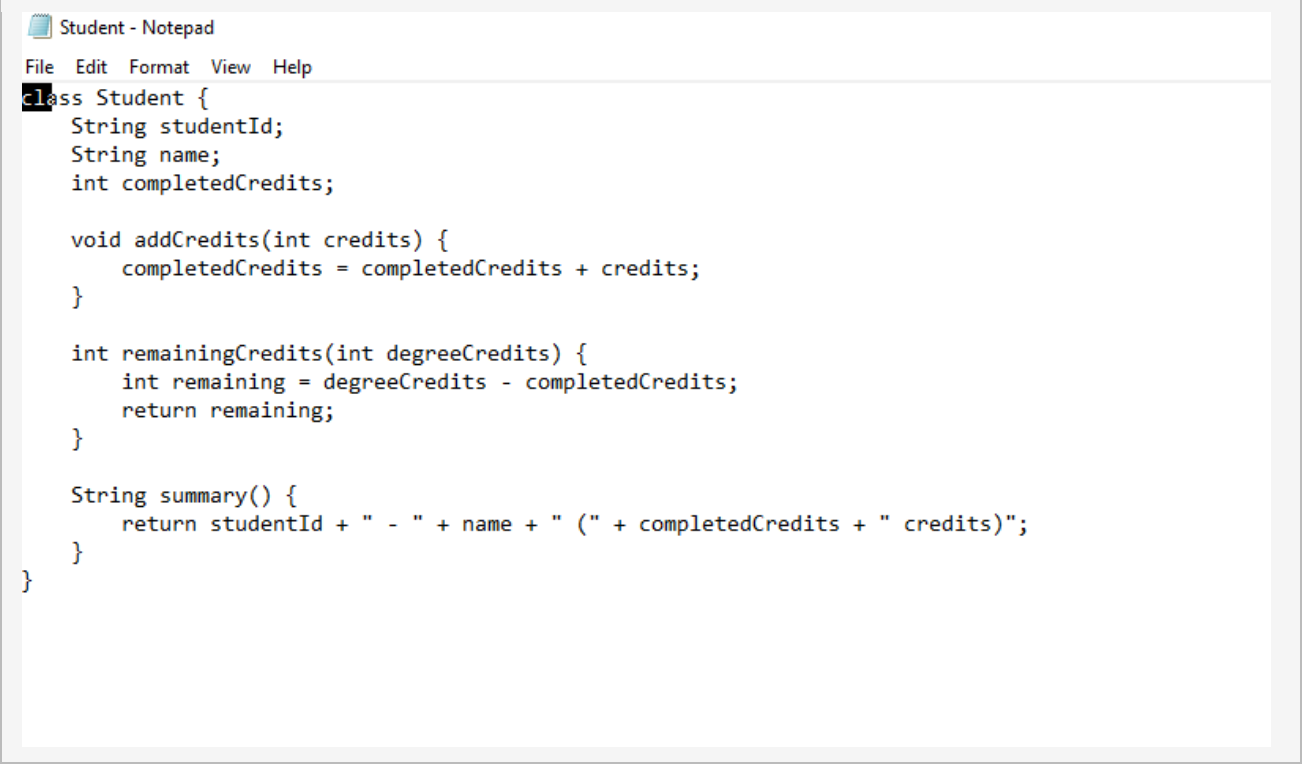
3. Which variables in Student represent persistent object state, and why are they instance variables rather than local variables?

Answer: studentId, name and completedCredits are the persistent state. They are declared as instance variables (inside the class, outside any method) so their values stay alive for as long as the object exists, instead of disappearing when a method finishes like a local variable would.

Task 2 - Add Behavior: Parameters, Local Variables, and Return Values

Student.java was extended with addCredits, remainingCredits and summary.

Student.java (updated)



```
Student - Notepad
File Edit Format View Help
class Student {
    String studentId;
    String name;
    int completedCredits;

    void addCredits(int credits) {
        completedCredits = completedCredits + credits;
    }

    int remainingCredits(int degreeCredits) {
        int remaining = degreeCredits - completedCredits;
        return remaining;
    }

    String summary() {
        return studentId + " - " + name + " (" + completedCredits + " credits)";
    }
}
```

Task2Demo.java

Task2Demo - Notepad

File Edit Format View Help

```
class Task2Demo {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.studentId = "SP26-BAI-001";
        s1.name = "Abeer Amina";
        s1.completedCredits = 15;

        Student s2 = new Student();
        s2.studentId = "SP26-BAI-002";
        s2.name = "Ali Ishtiaq";
        s2.completedCredits = 18;

        s1.addCredits(6);
        s2.addCredits(6);

        int s1Remaining = s1.remainingCredits(130);
        int s2Remaining = s2.remainingCredits(130);

        System.out.println(s1.summary());
        System.out.println("Remaining credits for s1: " + s1Remaining);
        System.out.println();
        System.out.println(s2.summary());
        System.out.println("Remaining credits for s2: " + s2Remaining);
    }
}
```

Output

```
SP26-BAI-001 - Abeer Amina (21 credits)  
Remaining credits for s1: 109
```

```
SP26-BAI-002 - Ali Ishtiaq (24 credits)  
Remaining credits for s2: 106
```

1. In addCredits(int credits), what is the difference between the parameter credits and the object field completedCredits?

Answer: credits is a parameter - a temporary value only available inside the method while it runs. completedCredits is a field that belongs to the object and keeps its value even after the method call ends.

2. Why is remaining inside remainingCredits(...) better as a local variable than as an instance field?

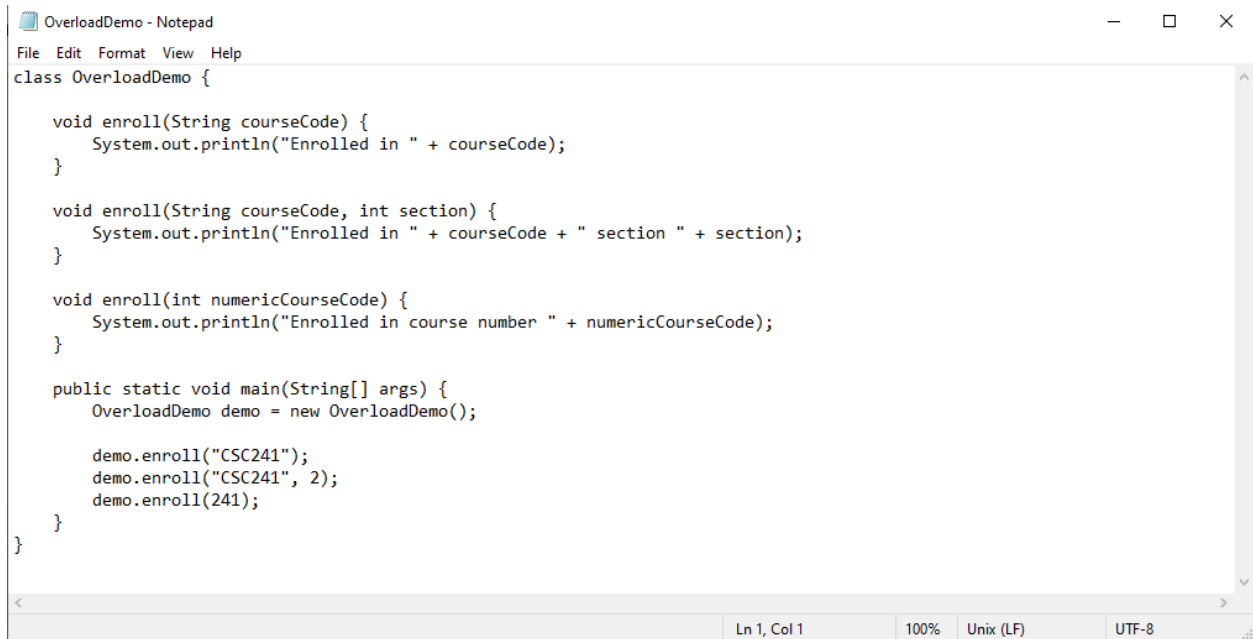
Answer: remaining is only needed to calculate the answer for that one method call, so it doesn't need to be remembered by the object afterward. Making it local keeps it temporary and avoids cluttering the object with data it doesn't actually need to store.

3. What information does the return type of remainingCredits(int degreeCredits) give to the caller?

Answer: The int return type tells the caller that calling this method will give back a whole number, which the caller can store in a variable or use right away, instead of the method just doing work silently like a void method.

Task 3 - Method Signatures and Overloading

OverloadDemo.java



```
class OverloadDemo {  
  
    void enroll(String courseCode) {  
        System.out.println("Enrolled in " + courseCode);  
    }  
  
    void enroll(String courseCode, int section) {  
        System.out.println("Enrolled in " + courseCode + " section " + section);  
    }  
  
    void enroll(int numericCourseCode) {  
        System.out.println("Enrolled in course number " + numericCourseCode);  
    }  
  
    public static void main(String[] args) {  
        OverloadDemo demo = new OverloadDemo();  
  
        demo.enroll("CSC241");  
        demo.enroll("CSC241", 2);  
        demo.enroll(241);  
    }  
}
```

Output

```
Enrolled in CSC241  
Enrolled in CSC241 section 2  
Enrolled in course number 241
```

1. What is the method signature of enroll(String courseCode, int section)?

Answer: enroll(String, int).

2. Why are enroll(String) and enroll(int) valid overloads, while two methods with the same name and parameter types but different return types are not?

Answer: Java tells overloads apart using only the method name and the parameter types/order - not the return type. enroll(String) and enroll(int) have different parameter types so they are fine, but two methods that only differ by return type look identical to the compiler and are rejected as duplicates.

3. For the call enroll("CSC241", 2), which overload should the compiler select, and what information does it use to make that decision?

Answer: The compiler picks enroll(String courseCode, int section) because it matches the number and types of the arguments (a String followed by an int) exactly.

Task 4 - Prove Java Pass-by-Value

PassByValueDemo.java

```
PassByValueDemo - Notepad
File Edit Format View Help
class PassByValueDemo {
    void changeNumber(int x) {
        System.out.println("Inside method (start): x = " + x);
        x = 99;
        System.out.println("Inside method (end): x = " + x);
    }

    void changeStudent(Student st) {
        System.out.println("Inside method (start): " + st.summary());
        st.completedCredits = 99;
        System.out.println("Inside method (end): " + st.summary());
    }

    void replaceStudent(Student st) {
        System.out.println("Inside method (start): " + st.summary());
        st = new Student();
        st.name = "Temporary";
        System.out.println("Inside method (end): " + st.summary());
    }

    public static void main(String[] args) {
        PassByValueDemo helper = new PassByValueDemo();

        int number = 10;
        System.out.println("Experiment A - primitive int");
        System.out.println("Before: number = " + number);
        helper.changeNumber(number);
        System.out.println("After: number = " + number);
        System.out.println();

        Student st = new Student();
        st.studentId = "SP26-BAI-001";
        st.name = "Abeer Amina";
        st.completedCredits = 15;

        System.out.println("Experiment B - mutate through reference");
        System.out.println("Before: " + st.summary());
        helper.changeStudent(st);
        System.out.println("After: " + st.summary());
        System.out.println("The argument value is copied into the parameter, but the copy still points at the same Student object.");
        System.out.println();

        System.out.println("Experiment C - reassign the parameter");
        System.out.println("Before: " + st.summary());
        helper.replaceStudent(st);
        System.out.println("After: " + st.summary());
        System.out.println("The argument value is copied into the parameter, so only the copy was redirected to the new object.");
    }
}
```

Output

```
Experiment A - primitive int
Before: number = 10
Inside method (start): x = 10
Inside method (end): x = 99
After: number = 10

Experiment B - mutate through reference
Before: SP26-BAI-001 - Abeer Amina (15 credits)
Inside method (start): SP26-BAI-001 - Abeer Amina (15 credits)
Inside method (end): SP26-BAI-001 - Abeer Amina (99 credits)
After: SP26-BAI-001 - Abeer Amina (99 credits)
The argument value is copied into the parameter, but the copy still points at the same Student object.

Experiment C - reassign the parameter
Before: SP26-BAI-001 - Abeer Amina (99 credits)
Inside method (start): SP26-BAI-001 - Abeer Amina (99 credits)
Inside method (end): null - Temporary (0 credits)
After: SP26-BAI-001 - Abeer Amina (99 credits)
The argument value is copied into the parameter, so only the copy was redirected to the new object.
```

Required explanation:

Why can changeStudent(...) affect the object, while replaceStudent(...) cannot redirect the caller variable?

changeStudent receives a copy of the reference, but that copy still points at the same Student object in memory, so using it to change a field changes the real object. replaceStudent also receives a copy of the reference, but pointing that copy at a brand new object only redirects the copy - it has no effect on the original variable back in main, which still points at the first object.

1. After changeNumber(int x) assigns x = 99, why does the caller's original int variable keep its old value?

Answer: x is a separate copy of the value that was passed in. Changing the copy inside the method has no way to reach back and change the original variable in main.

2. If a Student reference is passed to a method, why can changing st.completedCredits be visible to the caller?

Answer: The copied reference still points to the exact same object as the caller's variable, so following that reference to change a field changes the one real object that both are looking at.

3. Why does assigning st = new Student() inside a method not make the caller's variable refer to the new Student object?

Answer: st inside the method is only a local copy of the reference. Pointing that copy at a new object just changes what the copy points to - the caller's own variable is untouched and still points at the original object.

Task 5 - Integrated BAI Challenge: AIExperiment

AIExperiment.java

```
AIExperiment - Notepad
File Edit Format View Help
class AIExperiment {
    String experimentName;
    int completedEpochs;
    int targetEpochs;

    void runEpochs(int epochs) {
        completedEpochs = completedEpochs + epochs;
    }

    void runEpochs(int epochs, int bonusEpochs) {
        int totalEpochs = epochs + bonusEpochs;
        completedEpochs = completedEpochs + totalEpochs;
    }

    int remainingEpochs() {
        return targetEpochs - completedEpochs;
    }

    String status() {
        return experimentName + ": " + completedEpochs + "/" + targetEpochs + " epochs done";
    }
}
```

AIExperimentDemo.java

```
AIExperimentDemo - Notepad
File Edit Format View Help
class AIExperimentDemo {

    void boostExperiment(AIExperiment exp, int bonus) {
        exp.completedEpochs = exp.completedEpochs + bonus;
    }

    public static void main(String[] args) {
        AIExperimentDemo helper = new AIExperimentDemo();

        AIExperiment exp1 = new AIExperiment();
        exp1.experimentName = "ImageClassifier";
        exp1.completedEpochs = 0;
        exp1.targetEpochs = 20;

        AIExperiment exp2 = new AIExperiment();
        exp2.experimentName = "TextSummarizer";
        exp2.completedEpochs = 0;
        exp2.targetEpochs = 15;

        System.out.println("Initial state:");
        System.out.println(exp1.status());
        System.out.println(exp2.status());
        System.out.println();

        exp1.runEpochs(5);
        exp2.runEpochs(3, 2);

        System.out.println("After runEpochs calls:");
        System.out.println(exp1.status());
        System.out.println("Remaining for exp1: " + exp1.remainingEpochs());
        System.out.println(exp2.status());
        System.out.println("Remaining for exp2: " + exp2.remainingEpochs());
        System.out.println();

        System.out.println("Only exp1 changes below:");
        exp1.runEpochs(2);
        System.out.println(exp1.status());
        System.out.println(exp2.status());
        System.out.println();

        System.out.println("Before boostExperiment: " + exp1.status());
        helper.boostExperiment(exp1, 10);
        System.out.println("After boostExperiment: " + exp1.status());
        System.out.println("The change is visible because the copied reference still points at the same exp1 object.");
    }
}
```

Ln 1, Col 1 80% Unix (LF) UTF-8

Output

```
Initial state:
ImageClassifier: 0/20 epochs done
TextSummarizer: 0/15 epochs done

After runEpochs calls:
ImageClassifier: 5/20 epochs done
Remaining for exp1: 15
TextSummarizer: 5/15 epochs done
Remaining for exp2: 10

Only exp1 changes below:
ImageClassifier: 7/20 epochs done
TextSummarizer: 5/15 epochs done

Before boostExperiment: ImageClassifier: 7/20 epochs done
After boostExperiment: ImageClassifier: 17/20 epochs done
The change is visible because the copied reference still points at the same exp1 object.
```

1. In AIExperiment, which data should be stored as instance variables, and which calculations should remain local to methods?

Answer: experimentName, completedEpochs and targetEpochs are stored as instance variables because they describe the ongoing state of one experiment. Things like the combined epoch total in the overloaded runEpochs, or the remaining-epoch value, are just short calculations, so they stay as local variables/return values instead of being saved as fields.

2. Identify one method in your AIExperiment class that changes object state and one method that only computes or returns information.

Answer: runEpochs(int) changes state because it updates completedEpochs. remainingEpochs() only computes and returns a value (targetEpochs - completedEpochs) without changing anything.

3. If you created overloaded methods in this task, state their signatures and explain why the compiler can distinguish them.

Answer: runEpochs(int) and runEpochs(int, int) are the two overloads. The compiler can tell them apart because they take a different number of parameters, even though both parameters are of type int.

Five-Minute Exit Check

1. What is the difference between a class and an object?

Answer: A class is the design/template, and an object is an actual instance built from that template that holds real values in memory.

2. Why does changing `s1.completedCredits` not change `s2.completedCredits`?

Answer: `s1` and `s2` are separate objects, so each has its own independent copy of `completedCredits` stored at a different memory location.

3. What is the method signature of `enroll(String, int)`?

Answer: `enroll(String, int)`.

4. Why can two methods not be overloaded by return type alone?

Answer: Because the method signature only includes the name and parameter types - the compiler ignores the return type when deciding if two methods are different, so return type alone isn't enough to overload.

5. Why can a method mutate an object through a copied reference value but cannot reassign the caller's reference variable?

Answer: The copied reference still points at the same object, so changes made through it affect that real object. But reassigning the copy only changes what the copy points to, and has no effect on the original variable that still exists back in the caller.